



Java虚拟线程概述

北京理工大学计算机学院
金旭亮

技术背景



在互联网大行于世的时代，Web Server往往需要面临高并发的场景，人们需要尽可能地提升系统的并发性能和吞吐率，基于这个背景，响应式编程模型（Reactive Programming Model）应运而生，基于这种编程模型写出的代码，在运行时，是“异步非阻塞”的，在实际开发中得到了广泛的应用。



在JDK 21之前，JDK中提供了一些零散的组件，比如CompletableFuture和NIO库，使用它们编写异步非阻塞式的代码，但并没有提供一个现成的比较完备的编程框架，其结果就是在实际开发中，人们通常是使用RxJava，或者是Kotlin协程这样的框架来实现响应式编程。



等到JDK 21发布之后，Java终于将一个新特性集成到了JDK中，这个就是本课程要介绍的Java新特性——虚拟线程（Virtual Thread），使用它可以比较容易地编写异步响应式代码，从而在技术上追赶对手（比如Kotlin和Go支持的协程）。

复习一下JDK中CompletableFuture组件的用法

```
CompletableFuture.supplyAsync(  
    () -> {  
        HttpRequest request = ...;  
        return request;  
    })  
  
    .thenApply(  
        request -> {  
            var client = ...;  
            return client.send(request, ...);  
        })  
  
    .exceptionally(  
        throwable -> {...})  
  
    .thenApply(  
        response -> response.body())  
  
    .get();
```

左图所示为使用CompletableFuture典型的异步非阻塞编程模式，可以看到，数据处理逻辑分散在多个Lambda表达式中，然后通过一系列的特殊方法（thenApply）之类“串联”起来，这种编程方式，需要开发者在写代码前，仔细考虑清楚每一步干什么，上一步应该将什么信息传给下一步，.....，心智负担是挺重的。

这样的代码难于写单元测试代码，出了Bug也难以调试，在什么地方插入exceptionally进行异常捕获也是需要仔细斟酌，.....，其结果就是代码的可维护性受损。

RxJava vs. Virtual Thread



RxJava是Java平台使用最为广泛的响应式编程框架，以下是典型的RxJava代码。

```
private static void useFlowableToSlowDown() {  
    Flowable.range(1, 10000000)  
        .delay(10, TimeUnit.MILLISECONDS)  
        .map(MyItem::new)  
        .observeOn(Schedulers.io())  
        .subscribe(myItem -> {  
            sleep(50);  
            System.out.println("Received MyItem " + myItem.id);  
        });  
}
```

RxJava的功能比JDK中的CompletableFuture之类要强得多，但它存在的问题是API过于庞杂，学习曲线陡峭。



JDK 21所引入的虚拟线程，其API与传统的Thread高度一致，仅仅只是引入了少数的几个组件，就能方便地编写高性能的异步代码，比RxJava要易学易用得多，推荐在新项目中使用。

虚拟线程的卓越性能



操作系统创建一个本地线程，需要占用大约1M的内存空间，线程启动时间大约需要1毫秒，如果要进行线程上下文切换，则需要大约100微秒。

只需要简单地计算一下，如果一台Web Server需要处理多个并发连接，每个连接都创建一个线程，那么，创建100万个线程，将占用约1TB的内存，足以让一台Server因为资源耗尽而崩溃.....



而在JDK 21中，一个“平台（或本地）线程”可以对应多个虚拟线程，每个虚拟线程仅占用1K的内存，启动时间仅需1微秒，并且在很大程度上消除了“线程上下文切换”所带来的系统开销（因为在很多场景下仅需使用单线程就能实现异步执行的效果）。

通过以上介绍，可以看到，虚拟线程在高并发的场景下，是更好的选择。

线程池与虚拟线程之间的联系



JDK中集成了线程池，通过限制线程的数目，并且重用线程来解决反复创建、管理和销毁大量线程带来的问题。真实的系统中，基本上都是使用线程池来干活的。



线程池虽然高效，但在开发上与传统的单线程单任务是不一样的，需要采用不同的编程模式，具体细节参看“**Java异步与反应式编程入门**”专题课中的相应模块。



JDK 21所引入的“虚拟线程”，它在开发方式上，与单线程单任务是一样的，但在具体执行时，工作任务会被“分发”到一个后台线程池中去执行，这个“分发”过程是透明的，开发者无需操心。



“虚拟线程”的一大好处是，应用程序可以创建大量的虚拟线程（上百万级别的），而不需要担心占用大量的系统资源，因为它后面实际上是一个线程池，里面的线程可以创建多个虚拟线程，而线程池由于限制了本地线程创建的数目，因此，它所占用的资源是受限的、可控的，不用担心会由于创建过多的线程而耗尽系统资源。

关于本课程



本课程将介绍JDK 21所引入的虚拟线程技术特性，并通过典型示例展示其用法。



示例使用IntelliJ IDEA开发，以控制台程序为主，目的是避免细节淹没主题，让学习者将精力集中于技术特性和基础编程模式的把握之上。

掌握了上述知识，学习者自己就能轻松地在各种类型的Java项目中集成虚拟线程，开发出高性能的软件系统。

学习“Java虚拟线程”技术的前提



对Java面向对象与函数式编程有基本的了解



熟悉JDK中的集合和Stream API，以及IO库



对传统的Java多线程开发技术有基础了解：
线程与线程池，Runnable/Callable/Future接口，线程同步及
相关组件.....

让我们现在就开启一次新的技术之旅吧！